```
  # ...
end
```

*produces:*

```
Run options: --seed 18241
# Running:

...
Finished in 0.000445s, 6741.5728 runs/s, 103370.7836 assertions/s.
3 runs, 46 assertions, 0 failures, 0 errors, 0 skips
```

Each test starts by connecting to a database and creating a new playlist builder. Each test ends by disconnecting from the database. The idea of using a real database in unit tests is questionable, because unit tests are supposed to be fast running, context independent, and easy to set up, but it illustrates a point. (And that said, Ruby on Rails makes database calls in its unit tests all the time.)

We can extract all this common code into *setup* and *teardown* methods. Within a Minitest::Test class, if a method called setup exists, it will be run before each and every test method, and if a method called teardown exists, it will be run after each test method finishes. The setup and teardown methods bracket each test, rather than being run only once for the entire test case. This is shown in the code that follows:

unittesting/test_playlist_builder2.rb

```
require "minitest/autorun"
require_relative "playlist_builder"

class TestPlaylistBuilder < Minitest::Test
  def setup
    @database = DBI.new("DBI:mysql:playlists")
    @playlist_builder = PlaylistBuilder.new(@database)
  end

  def teardown
    @playlist_builder.close
  end

  def test_empty_playlist
    assert_empty(@playlist_builder.playlist)
  end

  def test_artist_playlist
    @playlist_builder.include_artist("krauss")
    refute_empty(@playlist_builder.playlist, "Playlist shouldn't be empty")
    @playlist_builder.playlist.each do |entry|
      assert_match(/krauss/i, entry.artist)
    end
  end

  def test_title_playlist
    @playlist_builder.include_title("midnight")
    refute_empty(@playlist_builder.playlist, "Playlist shouldn't be empty")
    @playlist_builder.playlist.each do |entry|
      assert_match(/midnight/i, entry.title)
    end
  end

  # ...
```